

API Test Report

Agentic Serial Story Writer

Expected results for every HTTP API

Document type: test specification with expected output.

Base URL: `http://localhost:8000`

Sample story: `634e82a1-9780-40bc-8b78-fde9b3dbe2f4`

Date: 29 September 2026

Routes are served twice: `/api/...` for the React UI, and the same handlers without `/api`.

How to execute

Run the cases in order. Plan generation and episode generation call the LLM and can take minutes. Prefer POST `/api/operations` for those two, then poll GET `/api/operations/{id}` until status is succeeded or failed. The synchronous POST `/plan` and POST `/episodes/generate` endpoints return the same final payload, but the HTTP call stays open until the agent finishes. Record Pass only when the status code and the listed JSON fields match.

Order of a full story test

1. Health and dashboard. 2. Create story. 3. List and get story. 4. Queue a plan (operation type=plan) and poll until succeeded. 5. GET plan. 6. PUT plan. 7. Approve plan (story status becomes active). 8. Resume (next_episode is 1). 9. Queue an episode and poll. 10. List and get the episode. 11. Approve, or edit, or reject. 12. Save feedback. 13. GET memory, feedback, and logs. 14. Optional: POST `/api/demo` on a fresh premise.

Known gate: episode generation returns 400 if stories.status is still planning, even when the plan row is approved. Approve plan sets status to active. The sample row was observed as planning with current_episode 0.

API-01 GET /health

Service is up.

Purpose

Confirm the API process is running before any story call.

Preconditions

Backend is started.

Request

```
GET http://localhost:8000/health
```

Expected status: 200

Expected response

```
{
  "status": "ok",
  "version": "0.1.0"
}
```

Expected failure

Connection refused if Uvicorn is not running. There is no JSON error body in that case.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-02 GET /api/dashboard

Dashboard counts.

Purpose

Load totals used by the home page.

Preconditions

Database is reachable.

Request

GET http://localhost:8000/api/dashboard

Expected status: 200

Expected response

```
{
  "total_stories": 1,
  "generating": 1,
  "completed": 0,
  "episodes_generated": 0,
  "recent_stories": [
    {
      "id": "634e82a1-9780-40bc-8b78-fde9b3dbe2f4",
      "title": null,
      "premise": "A delivery rider realizes every address...",
      "status": "planning",
      "current_episode": 0,
      "total_planned": 200
    }
  ],
  "recent_activity": []
}
```

Expected failure

500 if Postgres is down. generating counts stories whose status is planning or active.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-03 POST /api/stories

Create a story from a premise.

Purpose

Insert a story. This does not generate a plan.

Preconditions

Premise length is 10 to 2000 characters.

Request

POST http://localhost:8000/api/stories
Content-Type: application/json

```
{
  "premise": "A delivery rider realizes every address on today's route belongs to someone who died in the same building."
}
```

Expected status: 201

Expected response

```
{
  "id": "<uuid>",
  "premise": "A delivery rider realizes every address on today's route belongs to someone who died in the same building.",
  "status": "created"
}
```

Expected failure

422 if premise is shorter than 10 characters:

```
{ "detail": [ { "type": "string_too_short", "loc": [ "body", "premise" ], ... } ] }
```

Tester result: ☐ Pass ☐ Fail Notes: _____

API-04 GET /api/stories

List every story.

Purpose

Return the library used by the Stories page.

Preconditions

None. An empty database returns an empty array.

Request

```
GET http://localhost:8000/api/stories
```

Expected status: 200

Expected response

```
[
  {
    "id": "634e82a1-9780-40bc-8b78-fde9b3dbe2f4",
    "title": null,
    "premise": "<full premise>",
    "status": "planning",
    "current_episode": 0,
    "total_planned": 200,
    "created_at": "2026-09-29T11:38:54.305177+00:00",
    "updated_at": "2026-09-29T11:38:54.313863+00:00"
  }
]
```

Expected failure

200 with ☐ when no stories exist. Title stays null until a plan is saved.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-05 GET /api/stories/{story_id}

Read one story.

Purpose

Return story metadata for the workspace header.

Preconditions

Story id exists.

Request

```
GET http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4
```

Expected status: 200

Expected response

```
{
  "id": "634e82a1-9780-40bc-8b78-fde9b3dbe2f4",
  "title": "<plan title or null>",
  "premise": "<full premise>",
  "genre": "<genre or null>",
  "tone": "<tone or null>",
  "status": "planning",
  "current_episode": 0,
  "total_planned": 200,
  "created_at": "<iso8601>",
  "updated_at": "<iso8601>"
}
```

Expected failure

404 {"detail": "Story not found"} for an unknown id.

Tester result: [] Pass [] Fail Notes: _____

API-06 POST /api/operations

Generate the 200-episode plan in the background.

Purpose

Queue the planner. The UI uses this instead of waiting on the synchronous plan route.

Preconditions

Story exists. No other plan or episode job is already running for this story_id.

Request

POST http://localhost:8000/api/operations

Content-Type: application/json

```
{
  "type": "plan",
  "story_id": "634e82a1-9780-40bc-8b78-fde9b3dbe2f4"
}
```

Expected status: 202

Expected response

```
{
  "id": "<operation-uuid>",
  "type": "plan",
  "story_id": "634e82a1-9780-40bc-8b78-fde9b3dbe2f4",
  "status": "queued",
  "message": "Queued",
  "result": null,
  "error": null,
  "created_at": "<iso8601>",
  "updated_at": "<iso8601>"
}
```

Expected failure

422 {"detail": "story_id is required"} if story_id is omitted.

422 {"detail": "type must be plan, episode, or demo"} for any other type.

409 {"detail": "This story already has a generation job running"} if a job for this story is still queued or running.

Tester result: [] Pass [] Fail Notes: _____

API-07 GET /api/operations/{operation_id}

Poll a background job until it finishes.

Purpose

Read plan, episode, or demo progress.

Preconditions

operation_id came from API-06, API-13, or API-22. Poll about every 2 seconds.

Request

GET http://localhost:8000/api/operations/<operation-uuid>

Expected status: 200

Expected response

```
While running:
{
  "status": "running",
  "message": "Generating 200-episode plan...",
  "result": null,
  "error": null
}
```

```

}

When the plan succeeds, status is succeeded and result is:
{
  "plan_id": "<uuid>",
  "title": "<story title>",
  "episodes_planned": 200
}

When an episode succeeds, result is:
{
  "episode_number": 1,
  "episode_id": "<uuid>",
  "status": "human_review",
  "critic_passed": true,
  "critic_score": 0.86
}

```

Expected failure

404 {"detail": "Operation not found"} for an unknown id or after the API process restarts.

On failure the same 200 body has "status": "failed" and "error" set to the agent or validation message. A truncated planner response is a failed job, not HTTP 500.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-08 POST /api/stories/{story_id}/plan

Generate the plan on the request thread.

Purpose

Same planner as API-06, but the HTTP call blocks until the plan is stored.

Preconditions

Story exists. Expect a long response time.

Request

POST http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/plan

Expected status: 200

Expected response

```

{
  "plan_id": "<uuid>",
  "title": "<story title>",
  "episodes_planned": 200
}

```

Expected failure

400 {"detail": "Plan generation failed: ..."} if the model output cannot be parsed or the plan is incomplete.

400 {"detail": "Story <id> not found"} if the id does not exist.

After success, GET story status is plan_pending_review and title, genre, and tone are filled.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-09 GET /api/stories/{story_id}/plan

Read the stored plan.

Purpose

Return arcs, world rules, turning points, and all episode outlines.

Preconditions

A plan has been generated for the story.

Request

GET http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/plan

Expected status: 200

Expected response

```
{
  "id": "<plan uuid>",
  "story_id": "634e82a1-9780-40bc-8b78-fde9b3dbe2f4",
  "approved": false,
  "version": 2,
  "world_rules": ["The building has a dark history..."],
  "major_turning_points": ["Alex uncovers the first death..."],
  "planned_resolutions": [],
  "arc_structure": {
    "1": {
      "arc_number": 1,
      "title": "The First Delivery",
      "episode_start": 1,
      "episode_end": 20,
      "summary": "Alex starts his delivery route..."
    }
  },
  "episode_plans": [
    {
      "episode_number": 1,
      "title": "The Route Begins",
      "summary": "Alex Martin embarks on his delivery route...",
      "planned_hook": "As Alex leaves the building, he hears a faint whisper...",
      "arc_number": 1
    }
  ],
  "created_at": "<iso8601>"
}
```

Expected failure

404 {"detail": "Plan not found"} before the first successful plan.

episode_plans length must be 200 and arc_structure must cover episodes 1 through 200.

Tester result: [] Pass [] Fail Notes: _____

API-10 PUT /api/stories/{story_id}/plan

Edit plan fields without regenerating.

Purpose

Replace any supplied plan section. Omitted fields stay as they are.

Preconditions

A plan exists.

Request

PUT http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/plan
Content-Type: application/json

```
{
  "world_rules": [
    "The building has a dark history that connects all its residents.",
    "Ghosts can communicate with the living but cannot physically interact."
  ]
}
```

Expected status: 200

Expected response

```
{ "status": "updated" }
```

Expected failure

400 {"detail": "No plan found"} if no plan exists.

version increments by 1. approved is not cleared by this update.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-11 POST /api/stories/{story_id}/plan/approve

Approve the plan.

Purpose

Mark the plan approved and set the story to active so episodes can be generated.

Preconditions

A plan exists.

Request

POST http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/plan/approve

Expected status: 200

Expected response

```
{
  "status": "plan approved",
  "next": "POST /stories/{story_id}/episodes/next"
}
```

Expected failure

After this call, GET plan has approved true and GET story has status active. If status is still planning, episode generation will return 400.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-12 POST /api/stories/{story_id}/resume

Read the resume point.

Purpose

Report the last approved episode and the next number to generate. This does not generate prose.

Preconditions

Story exists.

Request

POST http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/resume

Expected status: 200

Expected response

```
{
  "story_id": "634e82a1-9780-40bc-8b78-fde9b3dbe2f4",
  "last_approved_episode": 0,
  "next_episode": 1,
  "status": "active",
  "title": "<story title>"
}
```

Expected failure

404 {"detail": "Story <id> not found"} for an unknown id.

After episode 1 is approved, last_approved_episode is 1 and next_episode is 2.

Tester result: [] Pass [] Fail Notes: _____

API-13 POST /api/operations

Queue the next episode.

Purpose

Write exactly one episode: last approved number plus one.

Preconditions

Plan is approved and story status is active or paused.

Request

POST http://localhost:8000/api/operations
Content-Type: application/json

```
{
  "type": "episode",
  "story_id": "634e82a1-9780-40bc-8b78-fde9b3dbe2f4"
}
```

Expected status: 202

Expected response

```
{
  "id": "<operation-uuid>",
  "type": "episode",
  "story_id": "634e82a1-9780-40bc-8b78-fde9b3dbe2f4",
  "status": "queued",
  "message": "Queued",
  "result": null,
  "error": null
}
```

Expected failure

Poll API-07. Failure error when the story is still planning:

"Story is not in active state: planning"

Also fails with "Story plan has not been approved yet" if approved is false.

Tester result: [] Pass [] Fail Notes: _____

API-14 POST /api/stories/{story_id}/episodes/generate

Generate the next episode on the request thread.

Purpose

Same writer and critic as API-13. Also registered as POST /episodes/next.

Preconditions

Plan approved. Story status active or paused. No episode already waiting in human_review for that number.

Request

POST http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/episodes/generate

Expected status: 200

Expected response

```
{
  "episode_number": 1,
  "episode_id": "<uuid>",
  "status": "human_review",
  "critic_passed": true,
  "critic_score": 0.86
}
```

Expected failure

400 {"detail": "Story is not in active state: planning"}

400 {"detail": "Story plan has not been approved yet"}

400 {"detail": "Story complete. All 200 episodes generated."} after episode 200 is approved.

If episode 1 is already in human_review, the body is {"episode_number": 1, "status": "already_in_human_review", "episode_id": "<uuid>"} and the model is not called again.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-15 GET /api/stories/{story_id}/episodes

List generated episodes.

Purpose

Sidebar list. Planned outlines that were never written are not included.

Preconditions

Story id can be any existing or unknown id; an unknown story returns an empty list.

Request

GET http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/episodes

Expected status: 200

Expected response

```
[
  {
    "episode_number": 1,
    "title": "The Route Begins",
    "status": "human_review",
    "word_count": 612,
    "critic_score": 0.86,
    "is_stale": false,
    "revision_count": 0
  }
]
```

Expected failure

200 ☐ before the first episode is generated. status is one of generating, human_review, approved, rejected, stale.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-16 GET /api/stories/{story_id}/episodes/{episode_number}

Read one generated episode.

Purpose

Full prose, critic issues, characters, and threads for the review screen.

Preconditions

That episode number has been generated.

Request

GET http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/episodes/1

Expected status: 200

Expected response

```
{
  "id": "<uuid>",
  "story_id": "634e82a1-9780-40bc-8b78-fde9b3dbe2f4",
  "episode_number": 1,
  "title": "The Route Begins",
  "content": "<400 to 700 words of prose>",
  "summary": "<2 to 3 sentences>",
  "hook": "<closing hook>",
  "status": "human_review",
  "word_count": 612,
  "revision_count": 0,
  "characters_present": ["Alex Martin"],
  "facts_introduced": [],
  "threads_opened": [],
  "threads_resolved": [],
  "critic_score": 0.86,
  "critic_issues": [
    {
      "type": "hook",
      "severity": "low",
      "description": "<issue text>"
    }
  ],
  "critic_passed": true,
  "human_edited": false,
  "rejection_reason": null,
  "is_stale": false,
  "stale_reason": null
}
```

Expected failure

404 {"detail": "Episode 1 not found"} if that number has not been generated. Episode plans are not returned by this route.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-17 POST /api/stories/{story_id}/episodes/{episode_number}/approve

Approve an episode and update memory.

Purpose

Mark the episode approved, set current_episode, and run the memory updater.

Preconditions

The episode exists.

Request

```
POST http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/episodes/1/approve
```

Expected status: 200

Expected response

```
{ "status": "approved", "episode_number": 1 }
```

Expected failure

400 {"detail": "Episode 1 not found"}.

After success, GET story current_episode is 1, episode status is approved, and GET memory includes the new summary, characters, facts, and threads.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-18 PUT /api/stories/{story_id}/episodes/{episode_number}

Save edited prose and approve it.

Purpose

Store human prose, set status approved, and mark later approved episodes stale.

Preconditions

The episode exists. content is required.

Request

```
PUT http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/episodes/1
```

```
Content-Type: application/json
```

```
{
  "content": "Alex Martin stopped at the first door. A pale mark sat above the number, and someone
  whispered his name.",
  "edit_notes": "Shortened the opening."
}
```

Expected status: 200

Expected response

```
{
  "status": "edited",
  "episode_number": 1,
  "word_count": 24
}
```

Expected failure

400 {"detail": "Episode 1 not found"}.

422 if content is missing. human_edited becomes true. Later approved episodes get is_stale true.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-19 POST /api/stories/{story_id}/episodes/{episode_number}/reject

Reject an episode.

Purpose

Mark the episode rejected. The next generate call rewrites the same number and receives this reason.

Preconditions

The episode exists. reason is required.

Request

```
POST http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/episodes/1/reject
Content-Type: application/json
```

```
{ "reason": "The whisper hook is missing from the last paragraph." }
```

Expected status: 200

Expected response

```
{ "status": "rejected", "episode_number": 1 }
```

Expected failure

400 {"detail": "Episode 1 not found"}.

422 if reason is missing. current_episode does not advance.

Tester result: [] Pass [] Fail Notes: _____

API-20 POST /api/stories/{story_id}/feedback

Save a persistent instruction for later episodes.

Purpose

Store an instruction that the writer receives from the next episode onward.

Preconditions

Story exists. instruction length is at least 5. episode is the episode this note follows.

Request

```
POST http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/feedback
Content-Type: application/json
```

```
{
  "episode": 3,
  "instruction": "Do not reveal the killer identity yet. Keep the mystery across later episodes."
}
```

Expected status: 201

Expected response

```
{
  "id": "<uuid>",
  "instruction": "Do not reveal the killer identity yet. Keep the mystery across later episodes.",
  "active": true,
  "will_affect_episodes": "Episode 4 onwards"
}
```

Expected failure

404 {"detail": "Story not found"}.

422 if instruction is shorter than 5 characters.

The same payload can be posted to /episodes/{episode_number}/feedback with body {"instruction": "..."}.

Tester result: [] Pass [] Fail Notes: _____

API-21 GET /api/stories/{story_id}/feedback

List feedback rows.

Purpose

Return approvals, edits, rejections, and active instructions.

Preconditions

Story exists.

Request

```
GET http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/feedback
```

Expected status: 200

Expected response

```
[
  {
    "id": "<uuid>",
    "episode": 3,
    "action": "feedback",
    "instruction": "Do not reveal the killer identity yet...",
    "rejection_reason": null,
    "is_active": true,
    "created_at": "<iso8601>"
  }
]
```

Expected failure

404 {"detail": "Story not found"}. An existing story with no notes returns [].

Tester result: [] Pass [] Fail Notes: _____

API-22 GET /api/stories/{story_id}/memory

Read story memory.

Purpose

Return the layered memory used for the next episode prompt.

Preconditions

None beyond a reachable database. Missing story still returns empty sections.

Request

```
GET http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/memory
```

Expected status: 200

Expected response

Before any approval:

```
{
  "rolling_summary": null,
  "characters": [],
  "world_facts": [],
  "open_threads": [],
  "active_instructions": []
}
```

After episode 1 is approved, characters and world_facts are objects, for example:

```
{
  "rolling_summary": "<compressed story so far>",
  "characters": [
    { "name": "Alex Martin", "role": "protagonist", "status": "alive", "current_state": "<state>" }
  ],
  "world_facts": [
    { "category": "location", "key": "building", "value": "<fact>" }
  ],
  "open_threads": [
```

```
{
  "title": "<thread>",
  "description": "<text>",
  "importance": "high"
},
"active_instructions": ["Do not reveal the killer identity yet..."]
}
```

Expected failure

active_instructions contains only rows with is_active true and a non-empty instruction. Approve and reject rows are not instructions.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-23 GET /api/stories/{story_id}/logs

Read agent logs and cost totals.

Purpose

Return planner, writer, critic, and memory-updater runs.

Preconditions

Story exists. Optional query limit defaults to 100.

Request

```
GET http://localhost:8000/api/stories/634e82a1-9780-40bc-8b78-fde9b3dbe2f4/logs?limit=20
```

Expected status: 200

Expected response

```
{
  "runs": [
    {
      "id": "<uuid>",
      "run_id": "<uuid>",
      "episode_number": 1,
      "agent": "writer",
      "model": "gpt-4o-mini",
      "status": "success",
      "input_tokens": 1800,
      "output_tokens": 900,
      "cost": 0.012,
      "latency_ms": 4200,
      "retry_count": 0,
      "created_at": "<iso8601>"
    }
  ],
  "summary": {
    "total_cost": 0.012,
    "total_input_tokens": 1800,
    "total_output_tokens": 900,
    "total_tokens": 2700,
    "total_runs": 1,
    "average_latency_ms": 4200
  }
}
```

Expected failure

404 {"detail": "Story not found"}.

Totals cover only the runs returned for the requested limit, not runs older than that page.

Tester result: ☐ Pass ☐ Fail Notes: _____

API-24 POST /api/demo

Run the automated demo in the background.

Purpose

Create a new story, generate and approve a plan, write episodes, and store the two sample instructions. Same body is accepted as POST /api/operations with type demo.

Preconditions

LLM key is configured. episodes is 1 to 30.

Request

POST http://localhost:8000/api/demo

Content-Type: application/json

```
{
  "premise": "A delivery rider realizes every address on today's route belongs to someone who died in the
same building.",
  "episodes": 1
}
```

Expected status: 202

Expected response

Immediate body:

```
{
  "id": "<operation-uuid>",
  "type": "demo",
  "story_id": null,
  "status": "queued",
  "message": "Queued",
  "result": null
}
```

Poll API-07. Succeeded result:

```
{
  "story_id": "<new uuid>",
  "title": "<plan title>",
  "episodes_generated": [1],
  "current_episode": 1,
  "characters": 2,
  "world_facts": 1,
  "open_threads": 1,
  "active_instructions": ["Do not reveal the killer's identity yet..."],
  "interventions": [
    {"after_episode": 3, "instruction": "Do not reveal the killer's identity yet..."}
  ],
  "total_runs": 4,
  "total_cost": 0.05
}
```

Expected failure

422 if episodes is greater than 30 or premise is shorter than 10 characters. The first instruction is stored even when fewer than 3 episodes are requested. The second instruction is stored only when episodes is at least 7.

Tester result: ☐ Pass ☐ Fail Notes: _____

Legacy paths

Every /api/stories route above also exists without the /api prefix, for example POST /stories and GET /stories/{id}/plan. Expected status and JSON are identical. /health, /api/dashboard, /api/operations, and /api/demo do not have an unprefixed alias except /health.

Pass rule

A case passes when the HTTP status matches and every field named in the expected body is present with the stated type. Prose, titles, scores, token counts, and UUIDs will differ between runs. Do not require the sample sentences to match exactly. A case fails when the status differs, a required field is missing, or episode generation is allowed while the story status is still planning.